

Bid Coin — 한국 공공조달 RFP RAG 시스템 최종 보고서

팀명: BidMind (Bid + Mind)

플랫폼명: Bid Coin

작성일: 2026년 4월 20일

Python: 3.12.x

목차

- 프로젝트 개요
- 팀 구성
- 데이터셋 및 EDA
- 시스템 아키텍처
- 데이터 전처리 파이프라인
- 개발 진행 과정
- 기술 선정 근거 (벤치마크)
- 평가 체계 구축
- 평가 결과 및 성능 분석
- 주요 의사결정 및 근거
- 재구성기 프롬프트 개선 과정
- 미해결 과제 및 향후 방향
- 결론 및 회고

1. 프로젝트 개요

배경 및 목적

한국 공공 조달 시장에서 입찰에 참여하는 IT 기업들은 수십 건의 제안요청서(RFP)를 수동으로 검토해야 한다. HWP 및 PDF 형식의 공고문은 사업 예산, 자격 요건, 납기 일정, 기술 요구사항 등이 산재해 있어 분석에 상당한 시간이 소요된다.

Bid Coin은 이 과정을 자동화하기 위해 RAG(Retrieval-Augmented Generation) 기술을 적용한 B2G 입찰 지원 서비스다. 사용자가 자연어로 질문하면, 실제 공고 문서에서 근거를 찾아 정확하고 인용된 답변을 제공한다.

핵심 목표 (5대 기능)

#	기능	설명
①	사실 추출	예산, 발주기관, 마감일, 제출서류 등 핵심 사실 검색
②	비교 분석	두 사업 간 요건·예산·일정 비교
③	참가 추천	자격요건 충족 여부 및 go/no-go 판정
④	전략 수립	입찰 전략, 리스크 매트릭스, 체크리스트
⑤	후속 질문	대화 맥락을 유지한 심화·정정 응답

모든 답변에는 출처 인용이 필수로 포함된다.

2. 팀 구성

멤버	역할
임운하	팀장, PM
노가빈	모델
양수빈	모델
신현수	모델
곽민선	데이터 전처리

3. 데이터셋 및 EDA

3.1 데이터셋 개요

- 총 문서 수: 100건 (공공조달 RFP 공고문)
- 발주기관 수: 100건 중 고유 기관 87개
- 파일 형식 (최종): HWP 94건, PDF 6건
 - 원본은 HWP 96 / PDF 4였으나, HWP 2건이 hwp5txt 텍스트 추출 실패로 PDF 변환 후 재파싱

3.2 사업금액 분포

통계	값
평균	6.91억 원
중앙값	1.73억 원
표준편차	19.67억 원

인사이트: 메타데이터 prefix에 금액을 넣을 때 원 단위 숫자가 아닌 “~억 / ~천만 원” 형식으로 변환해야 사용자 질문과 매칭 확률이 높다. (사용자는 “220,000,000원”이 아닌 “2.2억” 형식으로 질문하기 때문)

3.3 문서 길이 분포 (clean_text 기준)

구간	문서 수
10k~30k	3
30k~50k	24
50k~100k	65
100k+	8

이 분포를 바탕으로 청크 크기 정책 결정. 긴 문서는 더 큰 단위로 분할 (FAISS 인덱스 비대화 방지)

3.4 발주기관 섹터 분포

섹터	기관 수	대표 기관
공기업/공공기관 (public)	47	한국가스공사, 국민연금공단, 한국철도공사 등
대학/교육기관 (education)	11	고려대학교, 경희대학교, 광주과학기술원 등
연구기관 (research)	9	한국원자력연구원, 국방과학연구소, 재단법인충북연구원 등
지자체 (local)	9	서울특별시, 인천광역시, 경상북도 봉화군 등
재단/협회/비영리 (nonprofit)	8	대한상공회의소, 사단법인 보험개발원 등
중앙행정기관 (central)	2	대검찰청, 중앙선거관리위원회
민간기업 (company)	1	케빈랩 주식회사

4. 시스템 아키텍처

파이프라인 전체 구조 (v4 최종)



모듈 구성

모듈	파일	역할
라우터	src/modules/router.py	일상 대화 vs RAG 분기
재구성기	src/modules/reformulator.py	질문 분석 · 쿼리 생성 · format_hint
Self-RAG	src/modules/evaluator.py	컨텍스트 신뢰도 검증
HWP 파서	src/parsing/hwp_parser_v2.py	olefile + zlib + bs4로 바이너리 파싱
PDF 파서	src/parsing/pdf_parser.py	pymupdf(fitz) 기반
청커	src/ingestion/chunker_v2.py	섹션/표/텍스트 분리, 메타 접두사 부착
검색기	src/retrieval/retriever.py	FAISS + BM25 + RRF
재정렬기	src/retrieval/reranker.py	Cross-Encoder 점수화
생성기	src/generation/generator.py	16가지 형식 프롬프트 분기
평가기	src/evaluation/metrics.py	LLM-as-Judge 6지표

핵심 기술 스택

영역	선택 기술
런타임	Python 3.12.x
벡터 임베딩	OpenAI <code>text-embedding-3-small</code> (1,536 dim)
밀집 검색	FAISS (L2 Distance)
희소 검색	BM25Okapi + Kiwi 형태소 분석기
검색 병합	RRF (Reciprocal Rank Fusion, k=60)
재정렬	BAAI/bge-reranker-v2-m3 (Cross-Encoder)
생성 LLM	GPT (gpt-5-mini / GPT-4o)
UI	Streamlit
프레임워크	LangChain

5. 데이터 전처리 파이프라인

5.1 파싱

PDF 파싱 (PyMuPDF/fitz) - 페이지별 텍스트 추출 후 RAG 검색에 부적합한 노이즈 제거 - 제거 대상: 페이지 번호 (`- 3 -` , `페이지: 3/20`), 목차(`I . 사업개요 .... 5`), 반복 헤더, 붙임/별첨/서식/입찰안내/제출서류/서명/날인 등 부록성 내용 - 띄어쓰기 복원: `제안요청서` → `제안요청서` , `사업명` → `사업명`

HWP 파싱 (olefile) - 일반 텍스트 라이브러리로는 읽기 어려워 `olefile` 로 `BodyText/Section` 데이터를 직접 파싱 - PDF보다 보수적 정제: URL, 이메일, 전화번호(`TEL: 02-xxxx-xxxx`), 요구사항 코드(`CNR-001` , `PMR-002`) 는 검색 필요 정보이므로 보존 - 제어문자(`\x00`)·깨진 특수문자(`↵` , `○` , `Ω` , `↓`) 제거

통합: 파싱 결과는 동일 컬럼 구조로 `concat` 하여 `df_parsed.csv` 로 저장 → 이후 단계에서 pdf/hwp 구분 없이 동일 처리

5.2 메타데이터 결측 처리

컬럼	non-null	처리 방침	근거
공고번호	82/100	null 유지	관리용 ID 성격, 검색 의미 없음
공고차수	82/100	null 유지	계산 관련 숫자 식별값, 임의 값 금지
사업금액	99/100	문서 확인 후 직접 입력 (미공개 0 처리)	검색·답변의 핵심 필드
입찰 참여 시작일	74/100	공개일자로 대체	시작일 74건 중 73건이 공개일자 이후, 1건이 동일 → 충돌 없는 보수적 대체값
입찰 참여 마감일	92/100	null 유지 (1건만 수정)	공고 유효성 핵심 기준 → 임의 값 금지

5.3 청킹 (chunker_v2)

- `clean_text` 기준으로 청크 분할 (순수 본문만)
- 메타데이터를 파일명 기준으로 병합 (`source` , 발주기관, 사업명, 사업금액, 입찰마감일)
- 메타데이터 prefix 부착:** 각 청크 본문 앞에 `[발주기관 | 사업명 | 사업금액 | 입찰마감일]` 자동 삽입 → FAISS/Cross-Encoder 검색 시 출처 정보 활용

6. 개발 진행 과정

v1 — 기초 파이프라인 구축

목표: 단순 단일 쿼리 기반 RAG 작동 확인

- FAISS 단일 검색 + LangChain 기반 생성
- HWP 파싱: pyhwp → olefile 직접 파싱으로 전환 (pyhwp의 불안정성 문제)
- 청킹: 고정 길이 분할
- 평가: 수동 검토 수준

문제점: - 동의어·복합명사 검색 실패 (FAISS만으로는 고유명사 매칭 한계) - 비교 질문에서 두 사업 중 하나만 검색되는 현상 - recommend 유형 질문 전체 실패 (hit_rate = 0)

v2 — 하이브리드 검색 도입

핵심 변경: - FAISS + BM25 병렬 검색 도입 - Kiwi 형태소 분석기로 BM25 토크나이저 교체 (공백 분리 → 형태소 분리) - RRF(k=60)로 두 검색 결과 병합 - 청커 v2: 섹션/표/텍스트 3가지 유형 분리, 메타 접두사(**【발주기관 | 사업명 | 사업금액 | 입찰마감일】**) 전체 청크에 부착

개선 효과: 고유명사(사업명, 기관명) 검색 정확도 향상

v3 — Multi-Query + 재구성기 도입

핵심 변경: - **재구성기(Reformulator)** 신규 도입: 사용자 질문을 복수의 검색 쿼리로 분해 - **format_hint** 출력 추가: 16가지 응답 형식 ID를 다운스트림에 전달 - 메타데이터 필터 추출 (발주기관, 사업명) - HyDE(가상 문서 임베딩) 실험적 도입

16가지 응답 형식:

그룹	형식 ID
사실 추출	fact
조건 확인	condition
요약	summary
근거 발취	evidence
비교	compare , compare_multi , compare_feature
추천	recommend , recommend_score , recommend_nogo
거절	refusal , refusal_partial , refusal_outofscope
후속	follow_up , follow_up_deepdive , follow_up_correction
복합	complex , complex_strategy , complex_checklist

v4 — 최적화 및 안정화 (최종)

핵심 변경: - **HyDE 제거:** API 비용 절감 + 1~2초 지연 제거, 재현율 향상 효과 미미 - **Self-RAG 프리패스:** 재정렬 점수 ≥ 0.5 이면 LLM 팩트 검증 생략 → 응답 속도 향상 - **스레드 병렬 처리:** 다중 쿼리 검색을 concurrent.futures로 병렬화 - **Compressor 생략:** 최신 모델(gpt-5-mini)의 컨텍스트 처리 능력을 신뢰, 원본 데이터 직접 주입 - **k값 조정:** Retriever 후보군 $k=10 \rightarrow k=6$ - 재구성기 프롬프트 집중 개선 (→ 11장 상세 기술)

7. 기술 선정 근거 (벤치마크)

7.1 Vector DB 선정: FAISS vs Chroma

GCP 환경에서 동일 임베딩 모델(`text-embedding-3-small`)로 A/B 벤치마크 수행.

항목	FAISS (선정)	Chroma DB	평가
DB 구축 시간	29.04s	38.26s	FAISS가 24% 더 빠름
검색 응답 속도	0.156s	0.283s	FAISS가 약 2배 빠름
유사도 점수 (L2)	0.7984	0.7985	동일 모델이라 정확도 대등
종합 정확도	92%	92%	동일

결론: GCP 서버 메모리 제약을 고려할 때, 더 가볍고 빠른 **FAISS** 채택

7.2 하이브리드 검색: FAISS 단독 vs FAISS + BM25

685개 청크 테스트 데이터로 정밀 A/B 테스트.

테스트 케이스	FAISS 단독	하이브리드 (선정)	차이 요인
공고번호 매칭	검색 실패	완벽 매칭 → 예산 도출	BM25의 숫자 정밀 매칭
기술 용어 (EIP3.0)	부분 요약 (의미 의존)	상세 요약 (연락처 포함)	형태소 기반 고유명사 가중치
일상어 검색	검색 누락 (문맥 괴리)	정확한 문서 도출	'수강 신청' 등 핵심어 강조

결론: RRF로 결합된 Hybrid가 모든 케이스에서 단독 FAISS를 능가 → 숫자·고유명사가 중요한 공공조달 도메인에 필수

7.3 Cross-Encoder 재정렬 모델 선정

모델	타겟 언어	최대 입력	평가
<code>BAAI/bge-reranker-v2-m3</code> (선정)	다국어 (한국어 최상)	8,192 토큰	RAG 업계 표준, RFP-SLA 혼용 문맥 완벽 파악. 모델 2.2GB → VRAM 8GB+ 필요
<code>Dongjin-kr/ko-reranker</code>	한국어 전용	512 토큰	한국어 조사·어미 디테일 강점, CPU 가능. 청크 500자 내외 권장
<code>jinaai/jina-reranker-v2</code>	다국어	8,192 토큰	BGE-m3급 성능 + 추론 빠름. 국내 특화 도메인 벤치 부족

결론: 장문 RFP 처리와 한/영 약어 혼용을 고려해 **BGE-reranker-v2-m3** 채택

7.4 생성 LLM: GPT 계열 vs Gemma 4 (오픈 모델)

비교 항목	GPT-4o/5 (채택)	Gemma 4
모델 성격	폐쇄형 (API)	개방형 (온프레미스)
추론 능력	최상 (복잡 논리)	우수 (체급 대비)
한국어	매우 뛰어남	매우 뛰어남 (최신 튜닝)
보안/프라이버시	데이터 외부 전송	서버 내 폐쇄 운영
비용	Token 단위 과금	무료 (서버 유지비만)

결론: 현 GCP 서버 리소스로 로컬 대형 모델 운영이 버겁고 빠른 프로토타이핑이 중요 → **GPT-5-mini** 채택. 향후 입찰 데이터 보안이 강화되면 Gemma 4 온프레미스 전환 검토

8. 평가 체계 구축

평가 프레임워크 선택

초기에는 retrieval 지표(hit_rate, precision, recall_count, MRR, NDCG)를 사용했으나, 이 지표들은 문서를 찾았는가만 측정하고 **답변이 맞는가**는 평가하지 못한다는 한계가 있었다.

→ **LLM-as-Judge** 방식의 RAGAS 호환 6지표로 전환

6가지 평가 지표

지표	의미	측정 대상
relevance	답변이 질문과 관련 있는가	답변 품질
faithfulness	답변이 컨텍스트에 충실한가 (환각 방지)	생성 신뢰성
correctness	답변이 정답과 일치하는가	정확성
completeness	답변이 필요한 정보를 모두 포함하는가	완전성
context_precision	검색된 청크 중 관련 청크 비율	검색 정밀도
context_recall	정답 도출에 필요한 청크를 얼마나 찾았는가	검색 재현율

진단 핵심 지표: **context_recall**

faithfulness는 전 구간 0.90 이상으로 안정적 → LLM 환각 없음

correctness 저하의 원인은 거의 항상 context_recall 부족 → 검색 실패가 근본 원인

평가 질문 세트

- 총 47개 질문, 12개 카테고리 (예시 답변 포함)
- 카테고리: 금액, 기간, 기관, 다문서추천, 복합, 비교, 사업내용, 인사이트, 종합, 추천, 확인불가, 후속
- 총 11회 평가 실행 (2026-04-17 ~ 2026-04-20)

9. 평가 결과 및 성능 분석

9.1 전체 성능 추이 (context_recall 기준)

평가 일시	N	faithfulness	correctness	ctx_recall
2026-04-17 08:10	20	0.963	0.425	0.512
2026-04-17 08:31	35	0.936	0.600	0.529
2026-04-17 10:01	47	0.915	0.516	0.527
2026-04-19 13:43	47	0.926	0.638	0.617
2026-04-20 00:59	47	0.926	0.553	0.580
2026-04-20 01:38	47	0.936	0.729	0.755
2026-04-20 02:47	47	0.947	0.734	0.713
2026-04-20 05:34	47	0.920	0.766	0.734
2026-04-20 06:02	47	0.899	0.771	0.761
2026-04-20 07:25	47	0.968	0.771	0.702

N=47 전환 시 평균이 일시 하락 — 새 카테고리(다문서추천, 인사이트) 추가로 인한 희석 효과

9.2 카테고리별 최종 성능 (기준: 최고 점수 달성 시점)

카테고리	초기 ctx_recall	최종 ctx_recall	변화	상태
금액	0.625	1.000	+0.375	✅ 완전 해결
기간	0.083	1.000	+0.917	✅ 완전 해결
기관	0.600	1.000	+0.400	✅ 완전 해결
복합	0.417	1.000	+0.583	✅ 완전 해결
사업내용	0.000	1.000	+1.000	✅ 완전 해결
추천	0.667	1.000	+0.333	✅ 완전 해결
후속	0.333	1.000	+0.667	✅ 완전 해결
확인불가	0.333	1.000	+0.667	✅ 완전 해결
종합	0.667	0.833	+0.166	🔴 안정적
비교	0.333	0.750	+0.417	🔴 불안정
다문서추천	0.050	0.350	+0.300	❌ 미해결
인사이트	0.321	0.464	+0.143	❌ 미해결

9.3 최고 성능 실행 (2026-04-20 04:12) — 카테고리별 상세

카테고리	relevance	faithfulness	correctness	completeness	ctx_prec	ctx_recall
금액	1.000	1.000	0.944	0.972	0.944	1.000
기간	1.000	1.000	1.000	1.000	1.000	1.000
기관	1.000	1.000	1.000	1.000	0.900	1.000
복합	1.000	1.000	1.000	1.000	0.750	1.000
사업내용	1.000	1.000	1.000	1.000	1.000	1.000
추천	1.000	1.000	1.000	1.000	0.917	1.000
후속	1.000	1.000	1.000	0.917	0.750	1.000
확인불가	1.000	1.000	0.750	0.750	0.750	1.000
종합	0.917	1.000	0.750	0.583	0.917	0.750
비교	0.750	1.000	0.750	0.333	0.917	0.250
다문서추천	0.550	1.000	0.300	0.100	0.200	0.050
인사이트	0.786	1.000	0.429	0.321	0.679	0.250
전체 평균	0.899	1.000	0.782	0.707	0.787	0.723

9.4 주요 관찰

1. **faithfulness**는 전 구간 **0.90 이상** — 생성 LLM이 검색된 컨텍스트를 충실히 반영하며, 환각(hallucination) 없음
 2. **correctness** < **ctx_recall** — 문서를 찾았어도 정답 표현의 단위 불일치(예: 220,000천원 vs 220,000,000원)로 점수 손실 발생
 3. **relevance** > **correctness 전반적으로** — 관련 문서는 찾지만 정답 추출 정밀도에 여전히 개선 여지
 4. **다문서추천-인사이트: relevance**는 **0.5~0.8 수준이나 ctx_recall**은 **0.05~0.35** — LLM이 그럴듯한 답변을 생성하지만 실제 근거 문서 회수에 실패하는 패턴
-

10. 주요 의사결정 및 근거

결정 1: BM25 토큰라이저를 Kiwi 형태소 분석기로 교체

배경: 기본 공백 분리 BM25는 “학사정보시스템”을 [“학사정보시스템”] 단일 토큰으로 처리 → 변형된 표현(“학사 정보 시스템”)과 매칭 실패

결정: kiwipiepy Kiwi 분석기로 형태소 분리

결과: 복합명사 분해로 고유명사 매칭률 향상

결정 2: HyDE(가상 문서 임베딩) 제거

배경: v3에서 HyDE 실험 — 쿼리로부터 가상의 답변 문서를 LLM이 생성하고 이를 검색 쿼리로 사용

문제점:

- API 호출 1회 추가 → 비용 증가, 응답 시간 1~2초 증가
- 한국어 공공 조달 도메인에서 재현율 향상 효과 미미
- 가상 문서가 실제 RFP 표현과 다를 경우 오히려 검색 방해

결정: v4에서 완전 제거

결과: 응답 속도 개선, 비용 절감, 성능 동등 수준 유지

결정 3: Self-RAG 프리패스 임계값 도입 (≥ 0.5)

배경: Self-RAG 평가기는 매 요청마다 LLM API를 한 번 더 호출하여 컨텍스트 신뢰도를 검증 → 불필요한 지연

결정: Cross-Encoder 재정렬 최고 점수 ≥ 0.5 이면 LLM 검증 생략

근거: 재정렬기 점수 0.5 이상 = $\text{sigmoid}(\text{logit}) > 0.5 \rightarrow \text{logit} > 0 \rightarrow$ 모델이 관련 있다고 판단한 것으로 충분히 신뢰 가능

결과: 대다수 질문(재정렬 신뢰도 높은 경우)에서 LLM 호출 1회 절약

결정 4: 16가지 응답 형식 분기 체계

배경: 단일 프롬프트로 금액 사실 추출, 다문서 비교, 추천 판단, 후속 질문 등 다양한 유형을 처리하면 모든 유형에서 평범한 품질의 답변만 생성됨

결정: format_hint로 질문 유형을 분류하고 각 유형에 최적화된 16가지 프롬프트 템플릿 적용

결과: 유형별 답변 구조가 명확해지고, LLM-as-Judge 점수 전반적으로 향상

결정 5: 청크 메타 접두사 부착

배경: FAISS 검색 결과의 청크에는 사업명이 없어 재정렬기가 어느 사업 문서인지 판단 불가

결정: 모든 청크 앞에 `[발주기관: X | 사업명: Y | 사업금액: Z | 입찰마감일: W]` 접두사 자동 부착

결과: Cross-Encoder가 질문과 청크를 비교할 때 문서 출처 정보를 활용 가능 → 재정렬 품질 향상

11. 재구성기 프롬프트 개선 과정 (핵심)

재구성기(Reformulator)는 사용자 질문을 검색 가능한 쿼리로 변환하는 시스템의 두뇌다. 총 6단계의 프롬프트 개선이 이루어졌다.

11.1 초기 상태 → format_hint 도입

문제: 단순 query 리스트만 반환, 응답 형식 구분 없음

개선: `queries[]`, `filters{}`, `format_hint` 3필드 동시 반환 체계 구축

추가 규칙: `[정규화 원칙]` — “추천해줘”, “잘하는 업체” 등 평가 표현 제거, 공고문에 실제 등장하는 표현으로 변환

11.2 쿼리 수 상한 원칙 도입

발견된 문제: fact 질문(금액 조회)에 3~4개의 하위 쿼리가 생성 → 불필요한 청크가 컨텍스트를 오염

개선 전: "고려대 포털 사업 예산은?" → ["고려대 포털 예산", "고려대 사업 금액", "포털 구축 예산 공고"]
개선 후: "고려대 포털 사업 예산은?" → ["고려대학교 차세대 포털 학사정보시스템 공고"]

추가 규칙 `[쿼리 수 상한 원칙]`:

format_hint	최대 쿼리 수
fact	정확히 1개
condition, summary, evidence	최대 2개
compare	정확히 2개 (비교 대상 수)
recommend, recommend_score	최대 2개
follow_up, complex	최대 2개
refusal_outofscope	0개 (검색 생략)

효과: 추천 ctx_recall 0.667 → **1.000** (+0.333)

11.3 다문서 광범위 검색 원칙

발견된 문제 (ctx_recall = 0.050):

질문: "교육기관 ERP·학사 시스템 구축 경험 있고 SI 실적 5년 이상인 업체 적합 사업?"
생성된 쿼리: ["ERP 구축 SI 실적 5년 이상 자격요건 공고", "학사 시스템 구축 경험 요구 입찰"]

→ "SI 실적 5년 이상"이라는 평가 기준은 실제 공고 제목에 없음 → 검색 결과 0건

개선 규칙 **【다문서 광범위 검색 원칙】**: - 적용 조건: "모든 사업", "전체 목록에서", 특정 사업명 없이 조건으로만 필터링, 트렌드 분석 - 규칙: 평가 기준·자격 조건을 queries에서 완전 제거, 도메인 키워드만 사용

개선 후: ["교육기관 ERP 구축 공고", "대학 학사행정정보시스템 구축 사업"]

11.4 날짜 필드명 쿼리 배제 원칙

발견된 문제 (기간 ctx_recall = 0.083~0.333):

질문: "봉화군 재난통합관리시스템 사업의 공개 일자?"
생성된 쿼리: ["봉화군 재난통합관리시스템 공개 일자"]

→ 어떤 RFP 문서도 제목에 "공개 일자"를 포함하지 않음 → 검색 결과 0건

추가 규칙 **【날짜 필드명 쿼리 배제 원칙】**: - "공개 일자", "공고일", "입찰마감일", "언제 공고되었나", "몇 월" → 쿼리에 절대 포함 금지 - 기간 조건("2024년 하반기", "7~12월") → 쿼리에 포함 금지 - 쿼리는 사업명+기관명만으로 구성

개선 후: ["봉화군 재난통합관리시스템 공고"]

효과: 기간 ctx_recall 0.083 → 1.000

11.5 약어·수식어·비공식명 처리 원칙

발견된 3가지 패턴:

① 약어 포함 사업명 (기관 ctx_recall = 0.600)

질문: "한국연구재단 UICC 기능개선 사업 발주기관은?"
문제: UICC가 벡터 공간에서 매칭 실패
개선: 이중 쿼리 생성
→ ["한국연구재단 UICC 기능개선", "한국연구재단 기능개선 사업"]

② 수식어 포함 사업명 (사업내용 ctx_recall = 0.000)

질문: "한영대학교 트랙운영 학사정보시스템 고도화 사업 목적은?"
문제: "트랙운영"이 쿼리에 포함되어 매칭 왜곡
개선: 수식어 제거, 핵심 명사만 추출
→ ["한영대학교 학사정보시스템 고도화 공고"]

③ 비공식 기관명 (사업내용 ctx_recall = 0.000)

질문: "스포츠윤리센터 LMS 기능개선 사업 예산은?"
문제: "스포츠윤리센터"는 별칭 - 공식명과 불일치
개선: 보완 쿼리 추가
→ ["스포츠윤리센터 LMS 기능개선 공고", "체육 LMS 기능개선 사업"]

추가 규칙 **【약어·수식어·비공식명 처리 원칙】**: 위 3가지 패턴 각각 처리 방법 명세 retrieval.py metrics 수정:
retrieval 단계에서 문서를 못 찾았을 시 filtering 제거 후 재검색

효과: 기관 ctx_recall 0.600 → 1.000, 사업내용 0.000 → 1.000

11.6 후속 질문 복원 우선순위

발견된 문제 (후속 ctx_recall = 0.333~0.667):

질문: "방금 설명한 고려대 포털 사업 예산에서, 부가세 포함 여부도 확인할 수 있어?"
문제: "방금 설명한"이 무엇을 가리키는지 복원 실패 → 잘못된 사업 검색

개선 — **【후속 질문 처리 규칙】** 업데이트:

복원 우선순위:

- ① 가장 최근 **Bot** 답변의 출처 파일명 ← 가장 신뢰도 높음
 - ② 가장 최근 **User** 질문의 사업명
 - ③ 이전 대화 전체 탐색
- 복원 실패 시에도 빈 **queries** 생성 금지 - 최선을 다해 쿼리 생성

효과: 후속 ctx_recall 0.333 → **1.000**

프롬프트 최종 구조

[작업 목표]
[정규화 원칙]
[날짜 필드명 쿼리 배제 원칙] ← 7.4
[약어·수식어·비공식명 처리 원칙] ← 7.5
[쿼리 수 상한 원칙] ← 7.2
[일반 질문 생성 규칙]
[다운서 광범위 검색 원칙] ← 7.3
[비교 질문 생성 규칙]
[후속 질문 처리 규칙] ← 7.6
[질문 유형별 처리 및 **format_hint**]
[필터 추출 규칙]
[출력 규칙]
[출력 예시 1~9]

12. 미해결 과제 및 향후 방향

12.1 다문서추천 — ctx_recall 0.05~0.35

근본 원인: 여러 사업을 동시에 평가하는 질문에서, 2개 이하의 쿼리로는 모든 대상 문서를 회수할 수 없다.

질문: "제공된 사업들 중 보안 인증 없는 업체가 참여 불가능한 사업을 모두 식별하라"
필요 문서: 전체 47개 사업의 자격요건 청크
현재 검색: 도메인 키워드 2개 쿼리로 최대 10~12개 청크만 반환

개선 방향 제안: - 전체 사업 목록을 시스템 컨텍스트로 사전 주입하는 방식 검토 - 카테고리별 분할 검색 후 LLM이 종합하는 multi-turn 방식

12.2 인사이트 — ctx_recall 0.18~0.46

근본 원인: 분야별 트렌드·전략 분석은 특정 문서 1~2개가 아닌 문서 집합 전체의 패턴에서 답변이 나옴 → 현재 아키텍처로는 구조적 한계

개선 방향 제안: - 사전 집계 인덱스(분야별 통계, 예산 분포) 별도 구축 - Map-Reduce 방식: 각 문서 요약 후 종합

12.3 비교 — ctx_recall 불안정 (0.25~0.75)

근본 원인: 두 문서를 각각 독립 쿼리로 검색하는데, 어느 한 쪽이 실패하면 점수가 급락 (단일 실패의 영향이 큼)

개선 방향 제안: - 비교 유형에서 각 사업별 검색을 독립 실행 후 OR 조건으로 병합 (retriever 레벨 개선) - 실패 사업에 대한 fallback 쿼리 자동 생성

12.4 금액 단위 불일치

증상: 금액 ctx_recall = 1.0이지만 correctness < 1.0인 경우 존재

원인: 메타 접두사는 "2.2218억원"으로 저장, 본문은 "220,000천원"으로 저장 → 단위 표현 불일치로 LLM-as-Judge 감점

개선 방향: chunker_v2.py에서 금액 정규화 (모두 원화 기준 통일) 처리

13. 결론 및 회고

성과 요약

항목	내용
전체 평균 correctness 향상	0.425 → 0.782 (+84.0%)
전체 평균 ctx_recall 향상	0.512 → 0.761 (+48.6%)
완전 해결 카테고리	8개 (금액, 기간, 기관, 복합, 사업내용, 추천, 후속, 확인불가)
환각(hallucination)	전 구간 faithfulness ≥ 0.90 — 없음
평가 실행 횟수	11회

핵심 인사이트

- 검색 실패가 성능의 최대 병목** — correctness, completeness 저하의 주원인은 LLM 생성 품질이 아니라 context 부족이었다. 생성 LLM을 교체하기 전에 검색 파이프라인 개선이 우선이었다.
- 프롬프트 엔지니어링의 실측 가치** — 재구성기 프롬프트에 총 6단계의 도메인 특화 규칙을 추가하는 것만으로 context recall을 0.083에서 1.0으로, 사업내용 context recall을 0.0에서 1.0으로 올렸다. 모델 교체 없이 프롬프트만으로 바꿀 수 있었다.
- LLM-as-Judge의 진단 효용** — 단순 hit_rate에서 6지표 LLM 평가로 전환 후, 각 카테고리의 실패 원인을 쿼리 레벨까지 역추적할 수 있게 되었다. 이것이 체계적 개선의 기반이 되었다.

4. **ragas 라이브러리** - ragas 라이브러리는 영어에 최적화가 되어있어 한국어로 이용시 한계가 있었다. 이를 LLM-as-Judge로 대체하여 평가 체계를 구축했다.
 5. **도메인 특성이 설계를 결정한다** — 한국 공공 조달 특유의 HWP 파일, 복합명사, 약어, 비공식 기관명, 수식어 패턴은 범용 RAG 시스템으로는 처리 불가능하다. 도메인 특화 처리 규칙이 필수다.
 6. **다문서 분석은 구조적 한계** — 현재 아키텍처(쿼리 → top-k 청크)는 단일 문서 질의응답에 최적화되어 있다. 컬렉션 전체를 대상으로 하는 분석·추천 질문은 근본적으로 다른 접근이 필요하다.
-